

Generic Dictionary Collection Class in C#

In this article, I am going to discuss **Generic Dictionary<TKey, TValue> Collection Class in C#** with Examples. Please read our previous article where we discussed [Generic List<T> Collection Class in C#](#) with Examples. A Dictionary class is a data structure that represents a collection of keys and values pair of data. At the end of this article, you will understand the following pointers in detail.

1. [What is Dictionary in C#?](#)
2. [How to create a Generic Dictionary Collection in C#?](#)
3. [How to Add Elements into a Generic Dictionary<TKey, TValue> Collection in C#?](#)
4. [How to access a Generic Dictionary<TKey, TValue> Collection in C#?](#)
5. [How to Check the Availability of a key/value Pair in a Dictionary in C#?](#)
6. [How to Remove Elements from a Generic Dictionary Collection in C#?](#)
7. [How to Assign and Update Values to a Dictionary with Indexer in C#?](#)
8. [Generic Dictionary Collection with Complex Type in C#.](#)
9. [What is the use of the TryGetValue\(\) method of Dictionary Class in C#?](#)
10. [How to Convert an Array to a Dictionary in C#?](#)
11. [How to get all the keys and Values of a Dictionary in C#?](#)

What is Dictionary<TKey, TValue> Class in C#?

The Dictionary in C# is a Generic Collection that stores the element in the form of Key-Value Pairs. The working of the Generic Dictionary is very much similar to the working of the Non-Generic Hashtable collection. The difference is while creating the generic dictionary object we need to specify the type for both the keys as well as the type for values. As Dictionary<TKey, TValue> is a generic collection class, so it belongs to System.Collections.Generic namespace. The generic Dictionary<TKey, TValue> collection is dynamic in nature means the size of the dictionary is automatically increase as we added items to the collection. Following are a few points that you need to remember while working with Dictionary in C#.

1. In Generic Dictionary<TKey, TValue> Collection, the key cannot be null, but the value can be null if its type TValue is a reference type.
2. Every key in Generic Dictionary<TKey, TValue> Collection must be unique. Duplicate keys are not allowed. If you try to add a duplicate key then the compiler will throw an exception.
3. In Generic Dictionary<TKey, TValue> Collection, you can only store the same types of elements.
4. The capacity of a Dictionary collection is the number of elements that the Dictionary can hold.

How to create a Generic Dictionary Collection in C#?

The Generic Dictionary Collection class in C# provided the following constructors that we can use to create an instance of the Dictionary collection class.

1. **Dictionary():** It initializes a new instance of the Generic Dictionary class that is empty, has the default initial capacity and uses the default equality comparer for the key type.
2. **Dictionary(IDictionary<TKey, TValue> dictionary):** It initializes a new instance of the Generic Dictionary class that contains elements copied from the specified System.Collections.Generic.IDictionary and uses the default equality comparer for the key type.
3. **Dictionary(IEnumerable<KeyValuePair<TKey, TValue>> collection):** It initializes a new instance of the Generic Dictionary class that contains elements copied from the specified System.Collections.Generic.IDictionary and uses the default equality comparer for the key type.

4. **Dictionary(IEqualityComparer<TKey>? comparer):** It initializes a new instance of the Generic Dictionary class that is empty, has the default initial capacity, and uses the specified System.Collections.Generic.IEqualityComparer.
5. **Dictionary(int capacity):** It initializes a new instance of the Generic Dictionary class that is empty, has the specified initial capacity and uses the default equality comparer for the key type.
6. **Dictionary(IDictionary<TKey, TValue> dictionary, IEqualityComparer<TKey>? comparer):** It initializes a new instance of the Generic Dictionary class that contains elements copied from the specified System.Collections.Generic.IDictionary and uses the specified System.Collections.Generic.IEqualityComparer.
7. **Dictionary(int capacity, IEqualityComparer<TKey>? comparer):** It initializes a new instance of the Generic Dictionary class that is empty, has the specified initial capacity, and uses the specified System.Collections.Generic.IEqualityComparer.
8. **Dictionary(SerializationInfo info, StreamingContext context):** It initializes a new instance of the Generic Dictionary class with serialized data.

Let's see how to create an instance of the Dictionary collection class using the Dictionary() constructor in C#. The Dictionary() constructor is used to create an instance of the Dictionary class that is empty and has the default initial capacity.

Step1:

As the Dictionary<TKey, TValue> collection class belongs to System.Collections.Generic namespace, so first, we need to import the System.Collections.Generic namespace into our program as follows:

```
using System.Collections.Generic;
```

Step2:

Next, we need to create an instance of the Dictionary<TKey, TValue> class using the Dictionary() constructor as follows:

```
Dictionary<KeyDataType, ValuDataType> dictionary_name = new Dictionary<KeyDataType, ValuDataType>();
```

How to Add Elements into a Generic Dictionary<TKey, TValue> Collection in C#?

Now, if you want to add elements i.e. a key/value pair into the Dictionary, then you need to use the following Add() method of the Generic Dictionary Collection Class in C#.

1. **Add(TKey key, TValue value):** The Add(TKey key, TValue value) method is used to add an element with the specified key and value into the Dictionary. Here, the parameter key specifies the key of the element to add and the parameter value specifies the value of the element to add. The value can be null for a reference type but the Key cannot be null.

For Example:

```
Dictionary<string, string> dictionaryCountries = new Dictionary<string, string>();  
dictionaryCountries.Add("UK", "London, Manchester, Birmingham");  
dictionaryCountries.Add("USA", "Chicago, New York, Washington");  
dictionaryCountries.Add("IND", "Mumbai, Delhi, Bhubaneswar");
```

Even it is also possible to create a Dictionary<TKey, TValue> object using Collection Initializer syntax as follows:

```
Dictionary<string, string> dictionaryCountries = new Dictionary<string, string>  
{  
    {"UK", "London, Manchester, Birmingham"},
```

```

        {"USA", "Chicago, New York, Washington"},
        {"IND", "Mumbai, Delhi, Bhubaneswar"}
    };

```

How to access a Generic Dictionary<TKey, TValue> Collection in C#?

We can access the key/value pairs of the Dictionary<TKey, TValue> collection in C# using three different ways. They are as follows:

Using Key to access Dictionary<TKey, TValue> Collection in C#:

You can access the individual value of the Dictionary collection in C# by using the keys. In this case, we just need to specify the key to get the value from the given dictionary, no need to specify the index position. If the specified key is not present, then the compiler will throw an exception. The syntax is given below.

```

dictionaryCountries["UK"]
dictionaryCountries["USA"]

```

Using for-each loop to Access Dictionary<TKey,TValue> Collection in C#:

We can also use a for-each loop to access the key/value pairs of a Dictionary<TKey, TValue> in C# as follows.

```

foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
{
    Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
}

```

Using for loop to Access Dictionary<TKey, TValue> Collection in C#:

We can also access the Dictionary<TKey, TValue> collection in C# using a for loop as follows. The ElementAt method belongs to the static Enumerable class which is defined in **System.Linq** namespace. So, we need to include the **System.Linq** namespace in order to work the following code.

```

for (int i = 0; i < dictionaryCountries.Count; i++)
{
    string key = dictionaryCountries.Keys.ElementAt(i);
    string value = dictionaryCountries[key];
}

```

Example to Understand How to Create a Dictionary<TKey, TValue> Collection and Add Elements in C#:

For a better understanding of how to create a Dictionary<TKey, TValue> Collection and how to add elements to a Dictionary, and how to access the elements of a Dictionary in C#, please have a look at the below example.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericDictionaryDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Dictionary with Key and value both are string type

```

```

Dictionary<string, string> dictionaryCountries = new Dictionary<string, string>();
//Adding Elements to the Dictionary using Add Method of Dictionary
dictionaryCountries.Add("UK", "London, Manchester, Birmingham");
dictionaryCountries.Add("USA", "Chicago, New York, Washington");
dictionaryCountries.Add("IND", "Mumbai, Delhi, Bhubaneswar");
//Accessing Dictionary Elements using For Each Loop
Console.WriteLine("Accessing Dictionary Elements using For Each Loop");
foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
{
    Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
}
//Accessing Dictionary Elements using For Loop
Console.WriteLine("\nAccessing Dictionary Elements using For Loop");
for (int i = 0; i < dictionaryCountries.Count; i++)
{
    string key = dictionaryCountries.Keys.ElementAt(i);
    string value = dictionaryCountries[key];
    Console.WriteLine($"Key: {key}, Value: {value}");
}
//Accessing Dictionary Elements using Keys
Console.WriteLine("\nAccessing Dictionary Elements using Keys");
Console.WriteLine($"Key: UK, Value: {dictionaryCountries["UK"]}");
Console.WriteLine($"Key: USA, Value: {dictionaryCountries["USA"]}");
Console.WriteLine($"Key: IND, Value: {dictionaryCountries["IND"]}");
Console.ReadKey();
}
}
}

```

Output:

```

Accessing Dictionary Elements using For Each Loop
Key:UK, Value: London, Manchester, Birmingham
Key:USA, Value: Chicago, New York, Washington
Key:IND, Value: Mumbai, Delhi, Bhubaneswar

Accessing Dictionary Elements using For Loop
Key: UK, Value: London, Manchester, Birmingham
Key: USA, Value: Chicago, New York, Washington
Key: IND, Value: Mumbai, Delhi, Bhubaneswar

Accessing Dictionary Elements using Keys
Key: UK, Value: London, Manchester, Birmingham
Key: USA, Value: Chicago, New York, Washington
Key: IND, Value: Mumbai, Delhi, Bhubaneswar

```

Example to Add Elements to a Dictionary using Collection Initializer in C#:

This is a new feature added to C# 3.0 which allows initializing a collection directly at the time of declaration like an array. A Dictionary<TKey, TValue> contains a collection of key/value pairs. Its Add method takes two parameters, one for the key and one for the value. To initialize a Dictionary<TKey, TValue>, or any collection whose Add method takes multiple parameters, enclose each set of parameters in braces.

In the below example, we are using Collection Initializer syntax instead of the Add method of the Dictionary collection class to add key-value pairs into the dictionary object in C#.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericDictionaryDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Dictionary with Key and value both are string type using
            Collection Initializer
            Dictionary<string, string> dictionaryCountries = new Dictionary<string, string>
            {
                { "UK", "London, Manchester, Birmingham" },
                { "USA", "Chicago, New York, Washington" },
                { "IND", "Mumbai, Delhi, Bhubaneswar" }
            };
            //Accessing Dictionary Elements using For Each Loop
            Console.WriteLine("Accessing Dictionary Elements using For Each Loop");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Accessing Dictionary Elements using For Each Loop
Key:UK, Value: London, Manchester, Birmingham
Key:USA, Value: Chicago, New York, Washington
Key:IND, Value: Mumbai, Delhi, Bhubaneswar
```

How to Check the Availability of a key/value Pair in a Dictionary in C#?

If you want to check whether the key/value pair exists or not in the Dictionary collection, then you can use the following methods of the Generic Dictionary Collection Class in C#.

1. **ContainsKey(TKey key)**: The ContainsKey(TKey key) method of the Dictionary is used to check if the given key is present in the Dictionary or not. The parameter key to locating in the Dictionary object. If the given key is present in the collection, then it will return true else it will return false. If the key is null, then it will throw System.ArgumentNullException.
2. **ContainsValue(TValue value)**: The ContainsValue(TValue value) Method of the Dictionary class is used to check if the given value is present in the Dictionary or not. The parameter value to locate in the Dictionary object. If the given value is present in the collection, then it will return true else it will return false.

Let us understand this with an example. The following example shows how to use the ContainsKey and ContainsValue methods of the Generic Dictionary Collection class in C#.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Dictionary with Key and value both are string type us
Collection Initializer
            Dictionary<string, string> dictionaryCountries = new Dictionary<st
string>
            {
                { "UK", "United Kingdom" },
                { "USA", "United State of America" },
                { "IND", "India" }
            };
            //Accessing Dictionary Elements using For Each Loop
            Console.WriteLine("All Dictionary Elements");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }
            //Checking the key using the ContainsKey method
            Console.WriteLine("\nIs USA Key Exists : " +
dictionaryCountries.ContainsKey("USA"));
            Console.WriteLine("Is PAK Key Exists : " +
dictionaryCountries.ContainsKey("PAK"));
            //Checking the value using the ContainsValue method
            Console.WriteLine("\nIs India value Exists : " +
dictionaryCountries.ContainsValue("India"));
            Console.WriteLine("Is Srilanka value Exists : " +
dictionaryCountries.ContainsValue("Srilanka"));
            Console.ReadKey();
        }
    }
}
```

Output:

```
All Dictionary Elements
Key:UK, Value: United Kingdom
Key:USA, Value: United State of America
Key:IND, Value: India

Is USA Key Exists : True
Is PAK Key Exists : False

Is India value Exists : True
Is Srilanka value Exists : False
```

How to Remove Elements from a Generic Dictionary Collection in C#?

If you want to remove an element from the Dictionary, then you can use the following Remove method of the Dictionary collection class.

1. **Remove(TKey key):** This method is used to remove the element with the specified key from the Dictionary collection. Here, the parameter key specifies the element to remove. It throws `KeyNotFoundException` if the specified key is not found in the Dictionary, so check for an existing key using the `ContainsKey()` method before removing it.

If you want to remove all the elements from the Dictionary collection, then you need to use the following Clear method of the Dictionary class in C#.

1. **Clear():** This method is used to remove all elements i.e. all the keys and values from the Dictionary object.

For a better understanding of how to use the Remove and Clear method of the Generic Dictionary collection class, please have a look at the below example.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Dictionary with Key and value both are string type using
            Collection Initializer
            Dictionary<string, string> dictionaryCountries = new Dictionary<string,
            string>
            {
                { "UK", "United Kingdom" },
                { "USA", "United State of America" },
                { "IND", "India" },
            }
```

```

        { "PAK", "Pakistan" },
        { "SL", "Srilanka" }
    };
    Console.WriteLine($"Dictionary Elements Count Before Removing:
{dictionaryCountries.Count}");
    foreach (var item in dictionaryCountries)
    {
        Console.WriteLine($"Key:{item.Key}, Value:{item.Value}");
    }
    // Remove element PAK from Dictionary Using Remove() method
    if (dictionaryCountries.ContainsKey("PAK"))
    {
        dictionaryCountries.Remove("PAK");
        Console.WriteLine($"Dictionary Elements Count After Removing
{dictionaryCountries.Count}");
        foreach (var item in dictionaryCountries)
        {
            Console.WriteLine($"Key:{item.Key}, Value:{item.Value}");
        }
    }
    // Remove all Elements from Dictionary Using Clear method
    dictionaryCountries.Clear();
    Console.WriteLine($"Dictionary Elements Count After Clear:
{dictionaryCountries.Count}");
    Console.ReadKey();
}
}
}

```

Output:

```

Dictionary Elements Count Before Removing: 5
Key:UK, Value:United Kingdom
Key:USA, Value:United State of America
Key:IND, Value:India
Key:PAK, Value:Pakistan
Key:SL, Value:Srilanka

Dictionary Elements Count After Removing PAK: 4
Key:UK, Value:United Kingdom
Key:USA, Value:United State of America
Key:IND, Value:India
Key:SL, Value:Srilanka

Dictionary Elements Count After Clear: 0

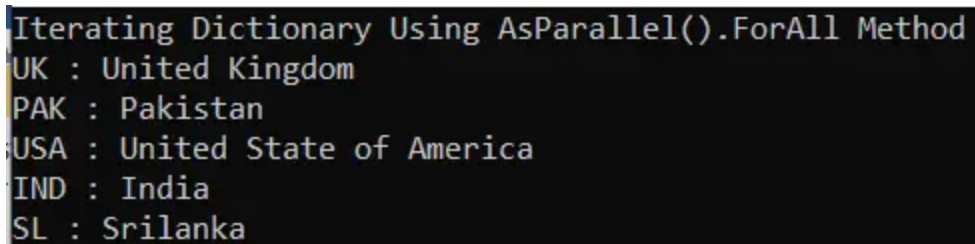
```

Using ParallelEnumerable.ForAll() Method to Iterate a Dictionary Collection in C#

Using `ParallelEnumerable.ForAll()` method is a simple but efficient way to iterate over large dictionaries. The following example shows how to iterate through a dictionary using `ParallelEnumerable.ForAll()` method.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Dictionary with Key and value both are string type using
            Collection Initializer
            Dictionary<string, string> dictionaryCountries = new Dictionary<string,
            string>
            {
                { "UK", "United Kingdom" },
                { "USA", "United State of America" },
                { "IND", "India" },
                { "PAK", "Pakistan" },
                { "SL", "Srilanka" }
            };
            Console.WriteLine($"Iterating Dictionary Using AsParallel().ForAll()");
            dictionaryCountries.AsParallel().ForAll(entry => Console.WriteLine(entry.Key + " : " + entry.Value));
            Console.ReadKey();
        }
    }
}
```

Output:



```
Iterating Dictionary Using AsParallel().ForAll Method
UK : United Kingdom
PAK : Pakistan
USA : United State of America
IND : India
SL : Srilanka
```

How to Assign Values to a Dictionary with Indexer in C#?

In order to add value to a Dictionary with an indexer, we need to use square brackets after the Dictionary name. This is because a Dictionary works with key/value pairs, and we have to specify both key and value while adding the elements. The key is specified between square brackets. The syntax is given below.

```
dictionary[key] = value;
```

For a better understanding, please have a look at the following example. In the below example, first, we have created the dictionary with a few key-value pairs. Then we added new key-value pair to the dictionaryCountries with the indexer. Here, IND, PAK, and SL are the keys, and India, Pakistan, and Srilanka are the values that correspond to each key respectively.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Dictionary with Key and value both are string type using
            Collection Initializer
            Dictionary<string, string> dictionaryCountries = new Dictionary<string,
            string>
            {
                { "UK", "United Kingdom" },
                { "USA", "United State of America" }
            };
            //Assign Values to a Dictionary with Indexer
            dictionaryCountries["IND"] = "India";
            dictionaryCountries["PAK"] = "Pakistan";
            dictionaryCountries["SL"] = "Srilanka";
            //Accessing the Dictionary using For Each Loop
            foreach (var item in dictionaryCountries)
            {
                Console.WriteLine($"Key:{item.Key}, Value:{item.Value}");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Key:UK, Value:United Kingdom
Key:USA, Value:United State of America
Key:IND, Value:India
Key:PAK, Value:Pakistan
Key:SL, Value:Srilanka
```

How to Update a Dictionary in C# using Indexer?

We already discussed that we can retrieve the value from the Dictionary by using the key in the indexer. In the same way, we can also use the key indexer to update an existing key-value pair in the Dictionary collection in C#. For a better understanding, please have a look at the below example.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            //Creating a Dictionary with Key and value both are string type using
            //Collection Initializer
            Dictionary<string, string> dictionaryCountries = new Dictionary<string, string>
            {
                { "UK", "United Kingdom" },
                { "USA", "United State of America" },
                { "IND", "India" },
                { "SL", "Srilanka" }
            };
            Console.WriteLine("Before Updating the Key UK and IND");
            Console.WriteLine($"USA: {dictionaryCountries["UK"]}");
            Console.WriteLine($"IND: {dictionaryCountries["IND"]}");
            //Updating the key UK and USA using Indexer
            dictionaryCountries["UK"] = "United Kingdom Updated";
            dictionaryCountries["IND"] = "India Updated";
            Console.WriteLine("\nAfter Updating the Key UK and IND");
            Console.WriteLine($"USA: {dictionaryCountries["UK"]}");
            Console.WriteLine($"IND: {dictionaryCountries["IND"]}");
            Console.ReadKey();
        }
    }
}

```

Output:

```

Before Updating the Key UK and IND
USA: United Kingdom
IND: India

After Updating the Key UK and IND
USA: United Kingdom Updated
IND: India Updated

```

Note: When accessing a dictionary value by key, make sure the dictionary contains the key, otherwise you will get a `KeyNotFoundException`.

Generic Dictionary Collection with Complex Type in C#:

As of now, we have used the built-in string and integer types with Dictionary. Now, let us see how to create a Dictionary collection with Complex types. For this, let us create a class called Student. Then create a Dictionary collection where the key is an integer which is nothing but the Id property of the student and the value is the Student type. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            Dictionary<int, Student> dictionaryStudents = new Dictionary<int,
            {
                { 101, new Student(){ ID = 101, Name = "Anurag", Branch="CSE"}
                { 102, new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"}
                { 103, new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}
                //The following Statement will give runtime error
                //System.ArgumentException: 'An item with the same key has alr
added. Key: 101'
                //{ 101, new Student(){ ID = 101, Name = "Anurag", Branch="CSE"
                };
                foreach (KeyValuePair<int, Student> item in dictionaryStudents)
                {
                    Console.WriteLine($"Key: {item.Key}, ID: {item.Value.ID}, Name
{item.Value.Name}, Branch: {item.Value.Branch}");
                }
                Console.ReadKey();
            }
        }
        public class Student
        {
            public int ID { get; set; }
            public string Name { get; set; }
            public string Branch { get; set; }
        }
    }
}
```

Output:

```
Key: 101, ID: 101, Name: Anurag, Branch: CSE
Key: 102, ID: 102, Name: Mohanty, Branch: CSE
Key: 103, ID: 103, Name: Sambit, Branch: ETC
```

What is the use of the TryGetValue() method of Dictionary Class in C#?

This is one of the important methods of Dictionary collection class in C#. This method takes two parameters, one is the key and the other one is the value. The value is of out type parameter. If the key exists in the dictionary, then it will return true and the value with that associated key is stored on the output variable.

If you are not sure if a key is present or not in the dictionary, then you can use the TryGetValue() method to get the value from a dictionary because if you are not using TryGetValue then in that case you will get KeyNotFoundException.

For a better understanding, please have a look at the below example. In the first TryGetValue method, we are passing the key as 102 and out variable i.e. std102. As we can see key 102 is present in the dictionary, so, this method will return true and the associated value will be populated in the std102 variable. And as the method returns true the body of the if condition gets executed and you can see the student data in the console window.

In the second TryGetValue method, we are passing the key as 105 and out variable i.e. std105. As we can see the key 105 is not present in the dictionary, so, this method will return false, and hence the value will not be populated in the std105 variable, and as the method returns false the else part of the if condition gets executed and that you can see in the console window.

```
using System;
using System.Collections.Generic;
namespace GenericDictionaryDemo
{
    class Program
    {
        static void Main()
        {
            Dictionary<int, Student> dictionaryStudents = new Dictionary<int,
            {
                { 101, new Student(){ ID = 101, Name = "Anurag", Branch="CSE"}
                { 102, new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"}
                { 103, new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}
            };
            foreach (KeyValuePair<int, Student> item in dictionaryStudents)
            {
                Console.WriteLine($"Key: {item.Key}, ID: {item.Value.ID}, Name
{item.Value.Name}, Branch: {item.Value.Branch}");
            }
            Student std102;
            if (dictionaryStudents.TryGetValue(102, out std102))
            {
                Console.WriteLine("\nStudent with Key = 102 is found in the di
                Console.WriteLine($"ID: {std102.ID}, Name: {std102.Name}, Brar
{std102.Branch}");
            }
            else
            {
                Console.WriteLine("\nStudent with Key = 102 is not found in th
dictionary");
            }
            Student std105;
```

```

        if (dictionaryStudents.TryGetValue(105, out std105))
        {
            Console.WriteLine("\nStudent with Key = 102 is found in the di
            Console.WriteLine($"ID: {std105.ID}, Name: {std105.Name}, Brar
{std105.Branch}");
        }
        else
        {
            Console.WriteLine("\nStudent with Key = 105 is not found in th
dictionary");
        }
        Console.ReadKey();
    }
}
public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}
}

```

Output:

```

Key: 101, ID: 101, Name: Anurag, Branch: CSE
Key: 102, ID: 102, Name: Mohanty, Branch: CSE
Key: 103, ID: 103, Name: Sambit, Branch: ETC

Student with Key = 102 is found in the dictionary
ID: 102, Name: Mohanty, Branch: CSE

Student with Key = 105 is not found in the dictionary

```

Note: If you are not sure if a key is present or not in the dictionary, then you can use the TryGetValue() method to get the value from a dictionary because if you are not using TryGetValue then in that case you will get KeyNotFoundException

How to Convert an Array to a Dictionary in C#?

To convert an array to a dictionary we need to use the ToDictionary() method. For a better understanding please have a look at the below example. In the below example, we are converting the Student array to a dictionary using the ToDictionary() method. Here, in the dictionary Key is Student ID and the value is the employee object itself.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericsDemo
{

```

```

class Program
{
    static void Main()
    {
        Student[] arrayStudents = new Student[3];
        arrayStudents[0] = new Student() { ID = 101, Name = "Anurag", Branch = "CSE" };
        arrayStudents[1] = new Student() { ID = 102, Name = "Mohanty", Branch = "CSE" };
        arrayStudents[2] = new Student() { ID = 103, Name = "Sambit", Branch = "ETC" };

        Dictionary<int, Student> dictionaryStudents = arrayStudents.ToDictionary(
            std => std.ID, std => std);
        // OR
        // Dictionary<int, Student> dictionaryStudents =
        arrayStudents.ToDictionary(employee => employee.ID);
        //OR use a foreach loop
        //Dictionary<int, Student> dict = new Dictionary<int, Student>();
        //foreach (Student std in arrayStudents)
        //{
        //    dict.Add(std.ID, std);
        //}
        foreach (KeyValuePair<int, Student> item in dictionaryStudents)
        {
            Console.WriteLine($"Key: {item.Key}, ID: {item.Value.ID}, Name: {item.Value.Name}, Branch: {item.Value.Branch}");
        }
        Console.ReadKey();
    }
}

public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}

```

Output:

```

Key: 101, ID: 101, Name: Anurag, Branch: CSE
Key: 102, ID: 102, Name: Mohanty, Branch: CSE
Key: 103, ID: 103, Name: Sambit, Branch: ETC

```

How to get all the keys and Values of a Dictionary in C#?

To get all the keys in the dictionary we have to use the Keys properties of the dictionary object. To get all the values of a dictionary, first, we need to get the keys, then we need to get the values using the keys. Even if

you only want the values, then you can use the Values property of the dictionary object. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace GenericsDemo
{
    class Program
    {
        static void Main()
        {
            Dictionary<int, Student> dictionaryStudents = new Dictionary<int,
            {
                { 101, new Student(){ ID = 101, Name = "Anurag", Branch="CSE"}
                { 102, new Student(){ ID = 102, Name = "Mohanty", Branch="CSE"}
                { 103, new Student(){ ID = 103, Name = "Sambit", Branch="ETC"}}
            };
            // To get all the keys in the dictionary use the keys properties of
dictionary
            Console.WriteLine("All Keys in Student Dictionary");
            foreach (int key in dictionaryStudents.Keys)
            {
                Console.WriteLine(key + " ");
            }

            // Once you get the keys, then get the values using the keys
            Console.WriteLine("\nAll Keys and values in Student Dictionary");
            foreach (int key in dictionaryStudents.Keys)
            {
                var student = dictionaryStudents[key];
                Console.WriteLine($"Key: {key}, ID: {student.ID}, Name: {student
Branch: {student.Branch}");
            }

            //To get all the values in the dictionary use Values property
            Console.WriteLine("\nAll Student objects in Student Dictionary");
            foreach (Student student in dictionaryStudents.Values)
            {
                Console.WriteLine($"ID: {student.ID}, Name: {student.Name}, Br
{student.Branch}");
            }
            Console.ReadKey();
        }
    }
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Branch { get; set; }
    }
}
```

```
}
```

Output:

```
All Keys in Student Dictionary
101
102
103

All Keys and values in Student Dictionary
Key: 101, ID: 101, Name: Anurag, Branch: CSE
Key: 102, ID: 102, Name: Mohanty, Branch: CSE
Key: 103, ID: 103, Name: Sambit, Branch: ETC

All Student objects in Student Dictionary
ID: 101, Name: Anurag, Branch: CSE
ID: 102, Name: Mohanty, Branch: CSE
ID: 103, Name: Sambit, Branch: ETC
```

Note: The Dictionary<TKey, TValue> generic class provides a mapping from a set of keys to a set of values. Each addition to the dictionary consists of a value and its associated key. Retrieving a value by using its key is very fast, close to O(1) because the Dictionary<TKey, TValue> class is implemented as a hash table which increases the performance of the application. The speed of retrieval depends on the quality of the hashing algorithm of the type specified for TKey.

C# Generic Dictionary Collection Class Summary:

1. A dictionary is a collection of key-value pairs.
2. The Dictionary Generic Collection class is present in System.Collections.Generic namespace.
3. When creating a dictionary, we need to specify the type for the key as well as the type for the value.
4. The fastest way to find a value in a dictionary is by using the keys.
5. Keys in a dictionary must be unique.

In the next article, I am going to discuss some important [Conversions Between Array, List, and Dictionary in C#](#) with Examples. Here, in this article, I try to explain **Generic Dictionary Collection Class in C#** with Examples. I hope this Generic Dictionary Collection Class in C# article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.